



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Compiler Design



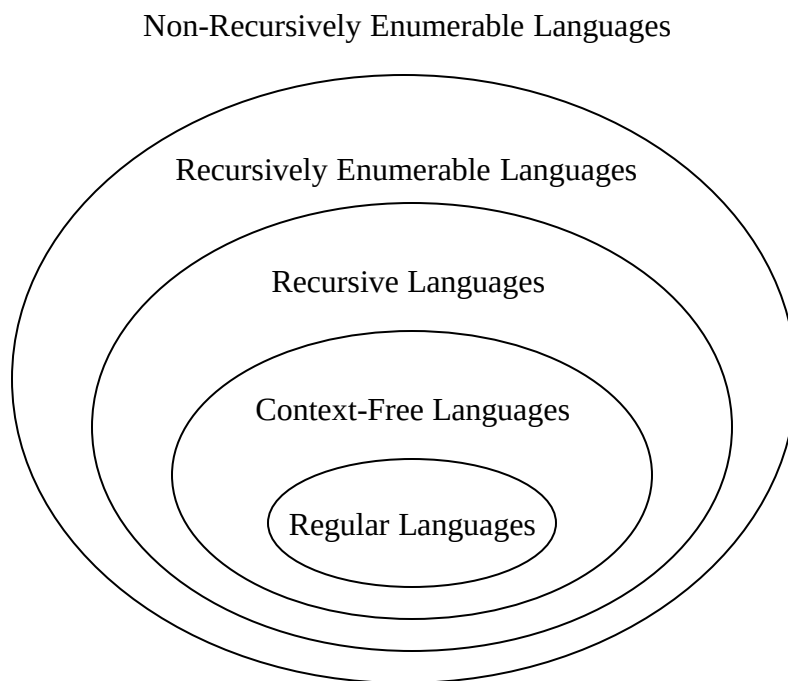
Contact Session - 3

Introduction to Lexical Analyzer

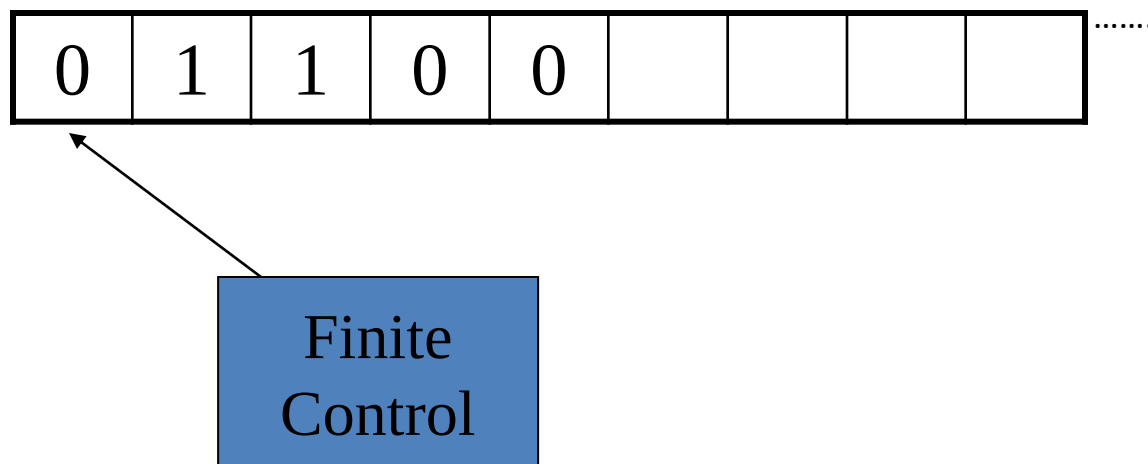
Objective

- Recap the Regular Expression
- Hierarchy of languages
- Deterministic Finite State Automata

Hierarchy of languages



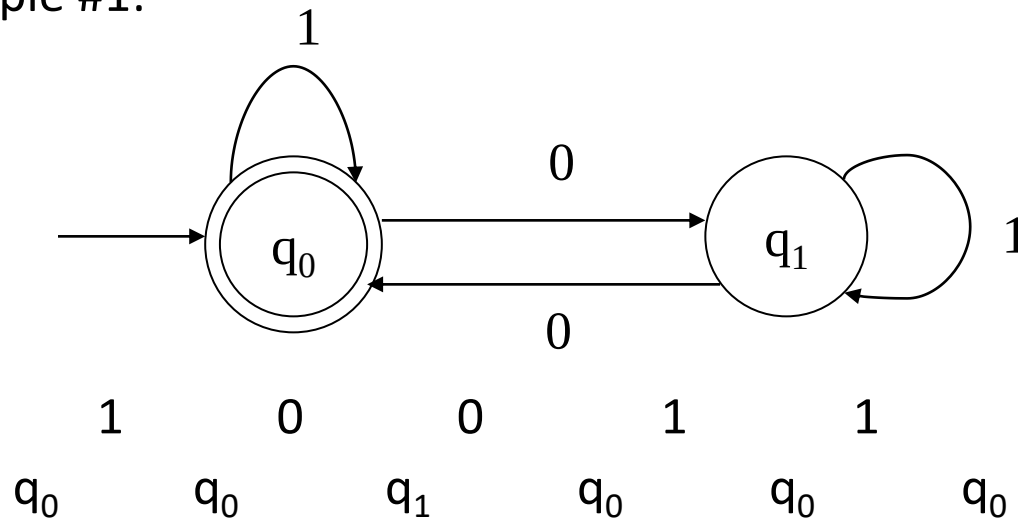
Deterministic Finite State Automata (DFA)



- One-way, infinite tape, broken into cells
- One-way, read-only tape head.
- Finite control, i.e.,
 - finite number of states, and
 - transition rules between them, i.e.,
 - a program, containing the position of the read head, current symbol being scanned, and the current “state.”
- A string is placed on the tape, read head is positioned at the left end, and the DFA will read the string one symbol at a time until all symbols have been read. The DFA will then either *accept* or *reject* the string.

- The finite control can be described by a transition diagram or table:

- Example #1:

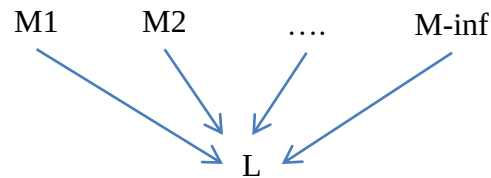


- One state is final/accepting, all others are rejecting.
- The above DFA accepts those strings that contain an even number of 0's, including the *null* string, over $\Sigma = \{0,1\}$

$$L = \{\text{all strings with zero or more 0's}\}$$
- Note, the DFA must reject all other strings

Note:

- *Machine is for accepting a language, language is the purpose!*
- *Many equivalent machines may accept the same language, but a machine cannot accept multiple languages!*

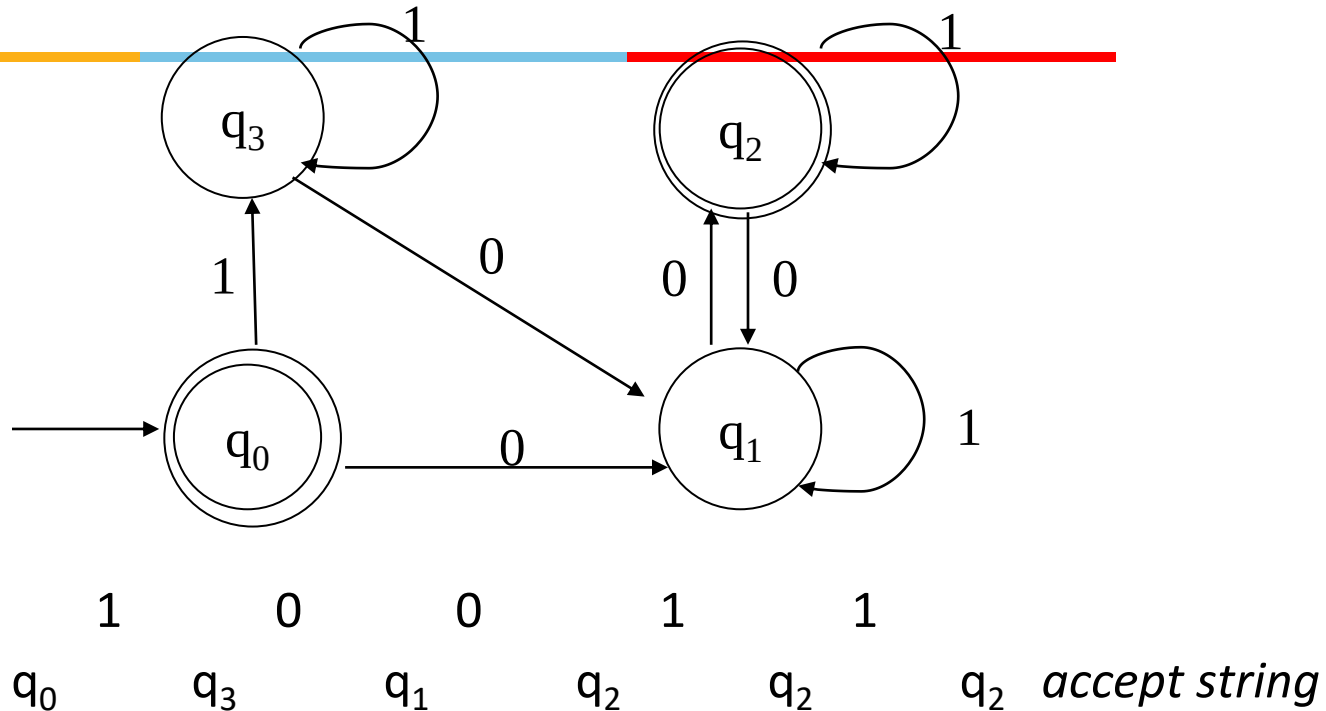


- *Id's of the characters or states are irrelevant, you can call them by any names!*

$\Sigma = \{0, 1\} \equiv \{a, b\}$

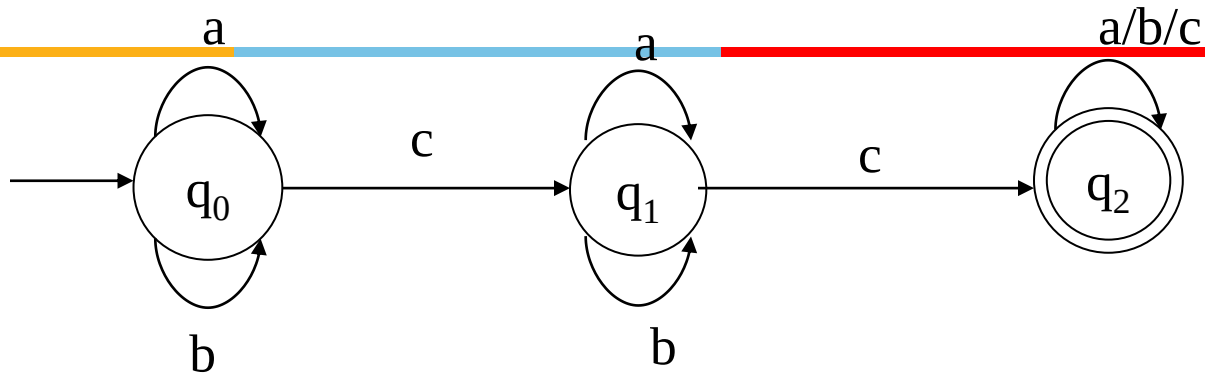
$States = \{q0, q1\} \equiv \{u, v\}$, as long as they have identical (isomorphic) transition table

- An equivalent machine to the previous example (DFA for even number of 0's):



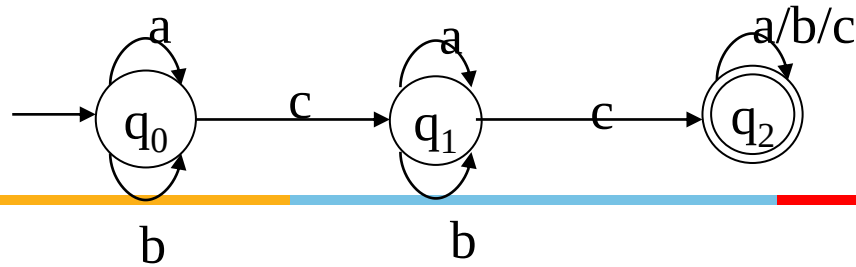
- One state is final/accepting, all others are rejecting.
- The above DFA accepts those strings that contain an even number of 0's, including null string, over $\Sigma = \{0,1\}$
- Can you draw a machine for a language by excluding the null string from the language? $L = \{\text{all strings with 2 or more 0's}\}$

- Example #2:



a	c	c	c	b	<u>accepted</u>
q ₀	q ₀	q ₁	q ₂	q ₂	q ₂
a	a	c			<u>rejected</u>
q ₀	q ₀	q ₀	q ₁		

- Accepts those strings that contain at least two *c*'s



Inductive Proof (sketch): that the machine correctly accepts strings with at least two c 's
Proof goes over the length of the string.

Base: x a string with $|x|=0$. state will be $q_0 \Rightarrow$ rejected.

Inductive hypothesis: $|x| = \text{integer } k$, & string x is *rejected* - in state q_0 (x must have zero c),
OR, rejected - in state q_1 (x must have one c),
OR, accepted - in state q_2 (x has already with two c 's)

Inductive steps: Each case for symbol p , for string xp ($|xp| = k+1$), the last symbol $p = a, b$ or c

	xa	xb	xc
x ends in q_0	$q_0 \Rightarrow \text{reject}$ (still zero $c \Rightarrow$ should reject)	$q_0 \Rightarrow \text{reject}$ (still zero $c \Rightarrow$ should reject)	$q_1 \Rightarrow \text{reject}$ (still zero $c \Rightarrow$ should reject)
x ends in q_1	$q_1 \Rightarrow \text{reject}$ (still one $c \Rightarrow$ should reject)	$q_1 \Rightarrow \text{reject}$ (still one $c \Rightarrow$ should reject)	$q_2 \Rightarrow \text{accept}$ (two c now $\Rightarrow$ should accept)
x ends in q_2	$q_2 \Rightarrow \text{accept}$ (two c already $\Rightarrow$ should accept)	$q_2 \Rightarrow \text{accept}$ (two c already $\Rightarrow$ should accept)	$q_2 \Rightarrow \text{accept}$ (two c already $\Rightarrow$ should accept)

Formal Definition of a DFA

- A DFA is a five-tuple:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q A finite set of states

Σ A finite input alphabet

q_0 The initial/starting state, q_0 is in Q

F A set of final/accepting states, which is a subset of Q

δ A transition function, which is a total function from $Q \times \Sigma$ to Q

$\delta: (Q \times \Sigma) \rightarrow Q$ δ is defined for any q in Q and s in Σ , and
 $\delta(q,s) = q'$ is equal to some state q' in Q , could be $q'=q$

Intuitively, $\delta(q,s)$ is the state entered by M after reading symbol s while in state q .

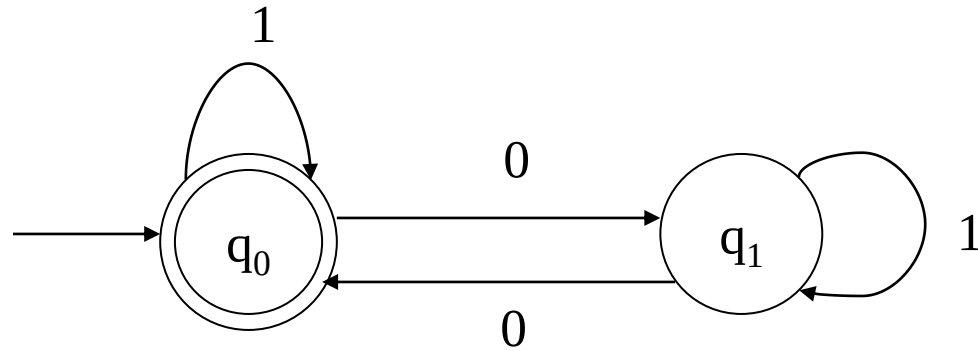
- Revisit example #1:

$Q = \{q_0, q_1\}$

$\Sigma = \{0, 1\}$

Start state is q_0

$F = \{q_0\}$



δ :

	0	1
q_0	q_1	q_0
q_1	q_0	q_1

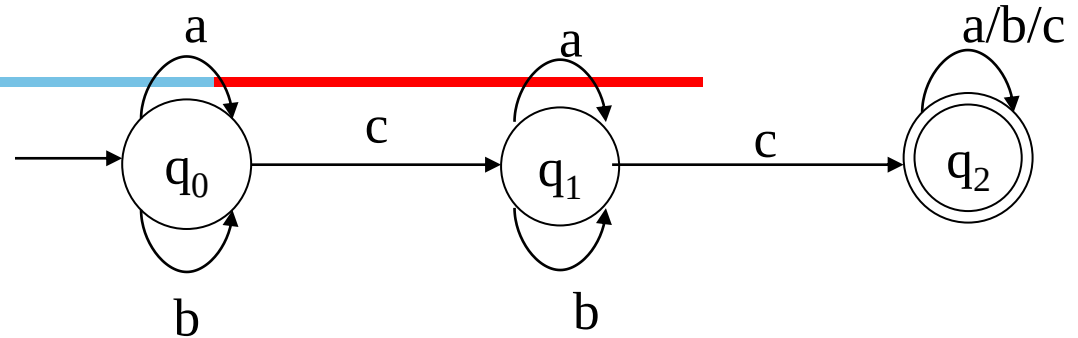
- Revisit example #2:

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{a, b, c\}$

Start state is q_0

$F = \{q_2\}$



δ :

	a	b	c
q_0	q_0	q_0	q_1
q_1	q_1	q_1	q_2
q_2	q_2	q_2	q_2

- Since δ is a function, at each step M has exactly one option.
- It follows that for a given string, there is exactly one computation.

Extension of δ to Strings

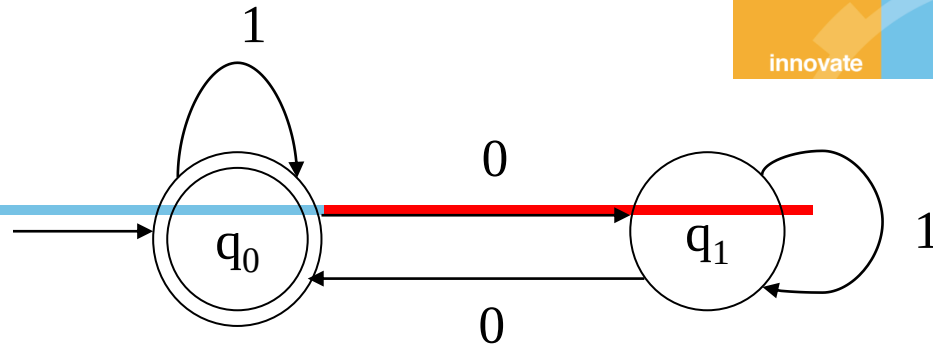
$$\delta^{\wedge} : (Q \times \Sigma^*) \rightarrow Q$$

$\delta^{\wedge}(q, w)$ – The state entered after reading string w having started in state q .

Formally:

- 1) $\delta^{\wedge}(q, \epsilon) = q$, and
- 2) For all w in Σ^* and a in Σ
$$\delta^{\wedge}(q, wa) = \delta(\delta^{\wedge}(q, w), a)$$

- Recall Example #1:



- What is $\delta^*(q_0, 011)$? Informally, it is the state entered by M after processing 011 having started in state q_0 .
- Formally:

$$\begin{aligned}
 \delta^*(q_0, 011) &= \delta(\delta^*(q_0, 01), 1) && \text{by rule \#2} \\
 &= \delta(\delta(\delta^*(q_0, 0), 1), 1) && \text{by rule \#2} \\
 &= \delta(\delta(\delta(\delta^*(q_0, \lambda), 0), 1), 1) && \text{by rule \#2} \\
 &= \delta(\delta(\delta(q_0, 0), 1), 1) && \text{by rule \#1} \\
 &= \delta(\delta(q_1, 1), 1) && \text{by definition of } \delta \\
 &= \delta(q_1, 1) && \text{by definition of } \delta \\
 &= q_1 && \text{by definition of } \delta
 \end{aligned}$$

- Is 011 accepted? No, since $\delta^*(q_0, 011) = q_1$ is not a final state.

- Note that:

$$\begin{aligned}\delta^{\wedge}(q, a) &= \delta(\delta^{\wedge}(q, \epsilon), a) && \text{by definition of } \delta^{\wedge}, \text{ rule \#2} \\ &= \delta(q, a) && \text{by definition of } \delta^{\wedge}, \text{ rule \#1}\end{aligned}$$

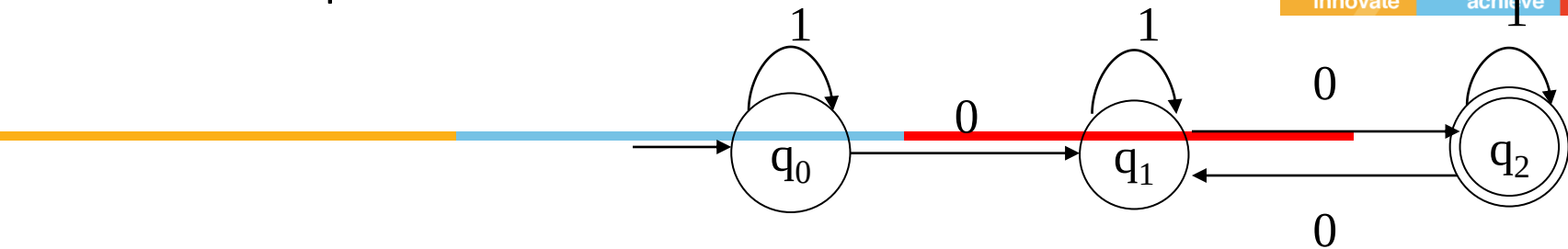
- Therefore:

$$\delta^{\wedge}(q, a_1 a_2 \dots a_n) = \delta(\delta(\dots \delta(\delta(q, a_1), a_2) \dots), a_n)$$

- However, we will abuse notations, and use δ in place of $\delta^{\wedge}$:

$$\delta^{\wedge}(q, a_1 a_2 \dots a_n) = \delta(q, a_1 a_2 \dots a_n)$$

- Example #3:

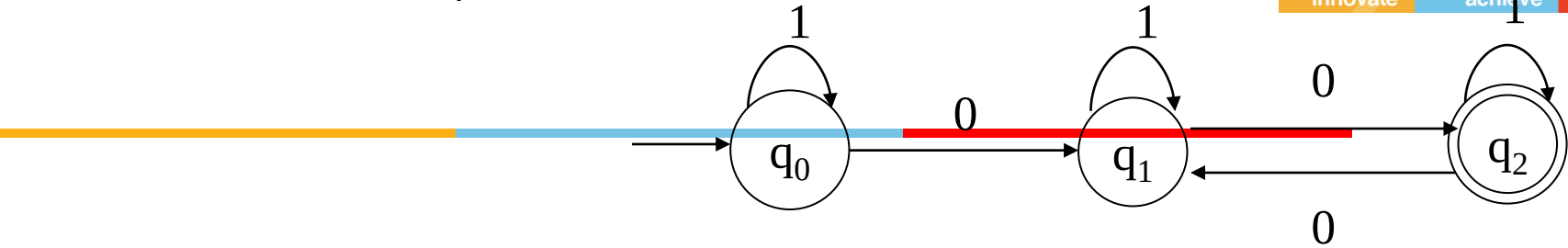


- What is $\delta(q_0, 011)$? Informally, it is the state entered by M after processing 011 having started in state q_0 .
- Formally:

$$\begin{aligned}
 \delta(q_0, 011) &= \delta(\delta(q_0, 01), 1) && \text{by rule \#2} \\
 &= \delta(\delta(\delta(q_0, 0), 1), 1) && \text{by rule \#2} \\
 &= \delta(\delta(q_1, 1), 1) && \text{by definition of } \delta \\
 &= \delta(q_1, 1) && \text{by definition of } \delta \\
 &= q_1 && \text{by definition of } \delta
 \end{aligned}$$

- Is 011 accepted? No, since $\delta(q_0, 011) = q_1$ is not a final state.
- Language?*
- $L = \{\text{all strings over } \{0,1\} \text{ that has 2 or more } 0 \text{ symbols}\}$

- Recall Example #3:



- What is $\delta(q_1, 10)$?

$$\begin{aligned}\delta(q_1, 10) &= \delta(\delta(q_1, 1), 0) \\ &= \delta(q_1, 0) \\ &= q_2\end{aligned}$$

by rule #2
by definition of δ
by definition of δ

- Is 10 accepted? No, since $\delta(q_0, 10) = q_1$ is not a final state. The fact that $\delta(q_1, 10) = q_2$ is irrelevant, q_1 is not the start state!

Definitions related to DFAs

- Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA and let w be in Σ^* . Then w is *accepted* by M if $\delta(q_0, w) = p$ for some state p in F .
- Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA. Then the *language accepted* by M is the set:

$$L(M) = \{w \mid w \text{ is in } \Sigma^* \text{ and } \delta(q_0, w) \text{ is in } F\}$$

- Another equivalent definition:

$$L(M) = \{w \mid w \text{ is in } \Sigma^* \text{ and } w \text{ is accepted by } M\}$$

- Let L be a language. Then L is a **regular language** iff there exists a DFA M such that $L = L(M)$.
- Let $M_1 = (Q_1, \Sigma_1, \delta_1, q_0, F_1)$ and $M_2 = (Q_2, \Sigma_2, \delta_2, p_0, F_2)$ be DFAs. Then M_1 and M_2 are *equivalent* iff $L(M_1) = L(M_2)$.

- Notes:

- A DFA $M = (Q, \Sigma, \delta, q_0, F)$ partitions the set Σ^* into two sets: $L(M)$ and $\Sigma^* - L(M)$.
- If $L = L(M)$ then L is a subset of $L(M)$ and $L(M)$ is a subset of L (def. of set equality).
- Similarly, if $L(M_1) = L(M_2)$ then $L(M_1)$ is a subset of $L(M_2)$ and $L(M_2)$ is a subset of $L(M_1)$.
- Some languages are regular, others are not. For example, if

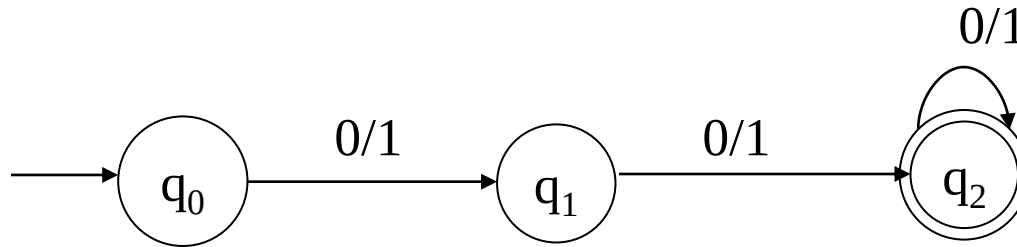
Regular: $L_1 = \{x \mid x \text{ is a string of 0's and 1's containing an even number of 1's}\}$ and

Not-regular: $L_2 = \{x \mid x = 0^n 1^n \text{ for some } n \geq 0\}$

- *Can you write a program to “simulate” a given DFA, or any arbitrary input DFA?*
- Question we will address later:
 - How do we determine whether or not a given language is regular?

- Give a DFA M such that:

$$L(M) = \{x \mid x \text{ is a string of 0's and 1's and } |x| \geq 2\}$$

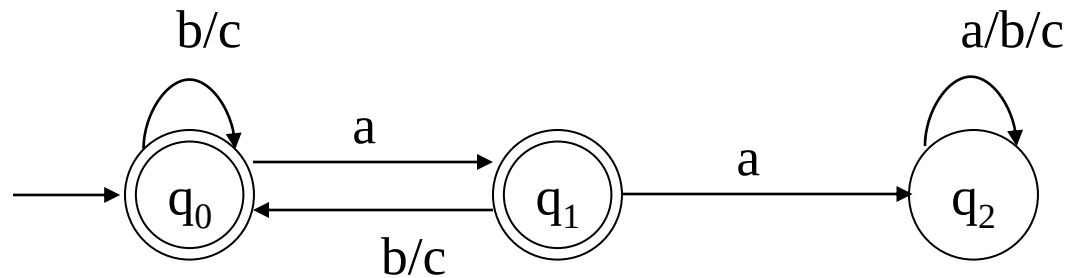


Prove this by induction

- Give a DFA M such that:



$L(M) = \{x \mid x \text{ is a string of (zero or more) } a\text{'s, } b\text{'s and } c\text{'s such that } x \text{ does not contain the substring } aa\}$



Logic:

In Start state (q0): b's and c's: ignore – stay in same state

q0 is also “accept” state

First ‘a’ appears: get ready (q1) to reject

But followed by a ‘b’ or ‘c’: go back to start state q0

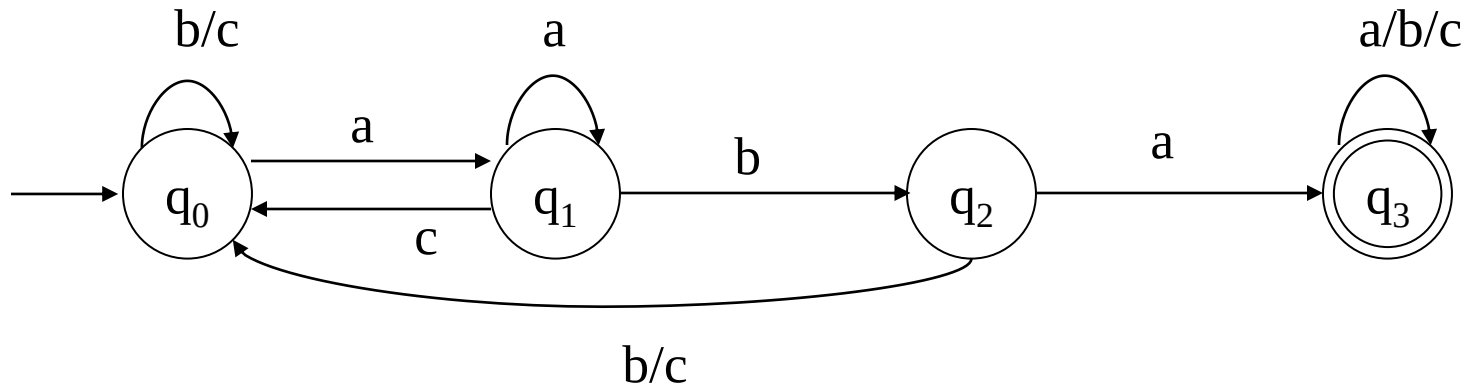
When second ‘a’ appears after the “ready” state: go to reject state q2

Ignore everything after getting to the “reject” state q2

- Give a DFA M such that:



$L(M) = \{x \mid x \text{ is a string of a's, b's and c's such that } x \text{ contains the substring } aba\}$



Logic: acceptance is straight forward, progressing on each expected symbol

However, rejection needs special care, in each state (for DFA, we will see this becomes easier in NFA, non-deterministic machine)

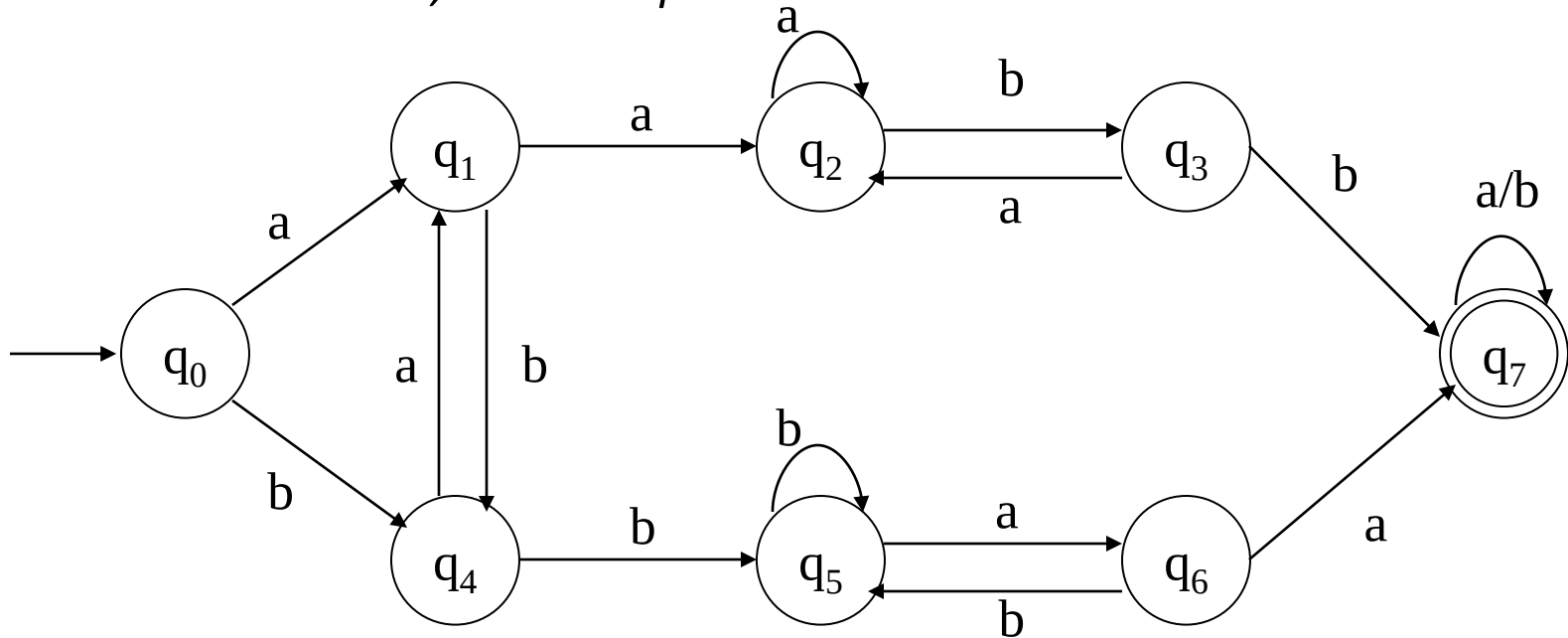
- Give a DFA M such that:



$L(M) = \{x \mid x \text{ is a string of a's and b's such that } x \text{ contains both } aa \text{ and } bb\}$

First do, for a language where 'aa' comes before 'bb'

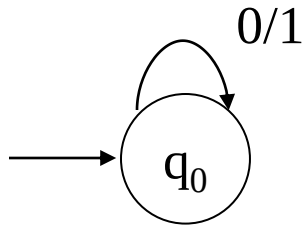
Then do its reverse; and then parallelize them.



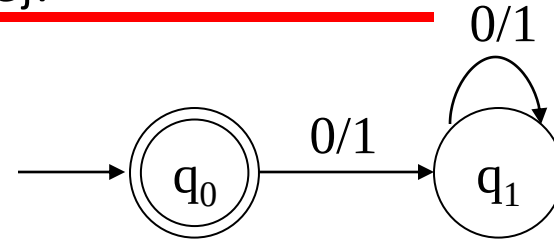
Remember, you may have multiple “final” states, but only one “start” state

- Let $\Sigma = \{0, 1\}$. Give DFAs for $\{\}$, $\{\epsilon\}$, Σ^* , and Σ^+ .

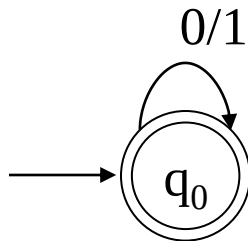
For $\{\}$:



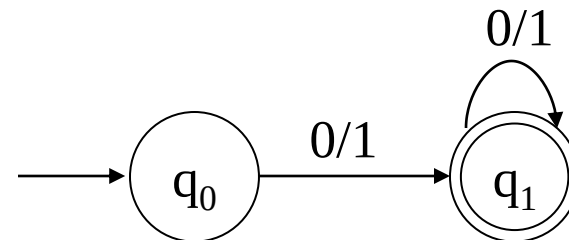
For $\{\epsilon\}$:



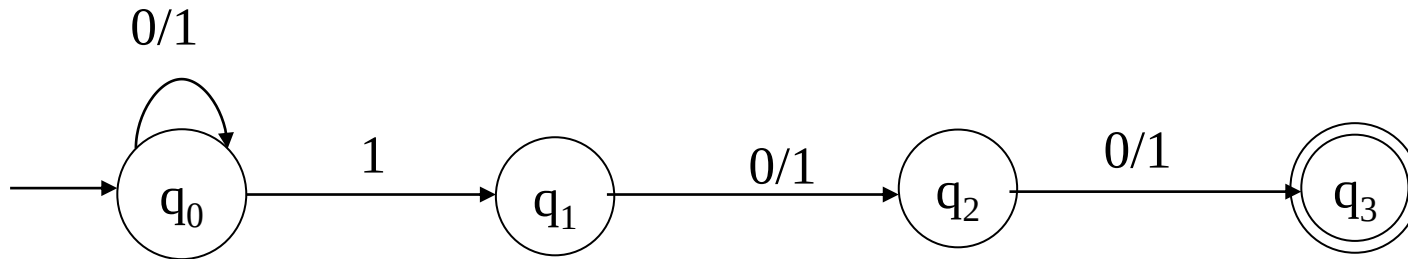
For Σ^* :



For Σ^+ :



- Problem: Third symbol from last is 1



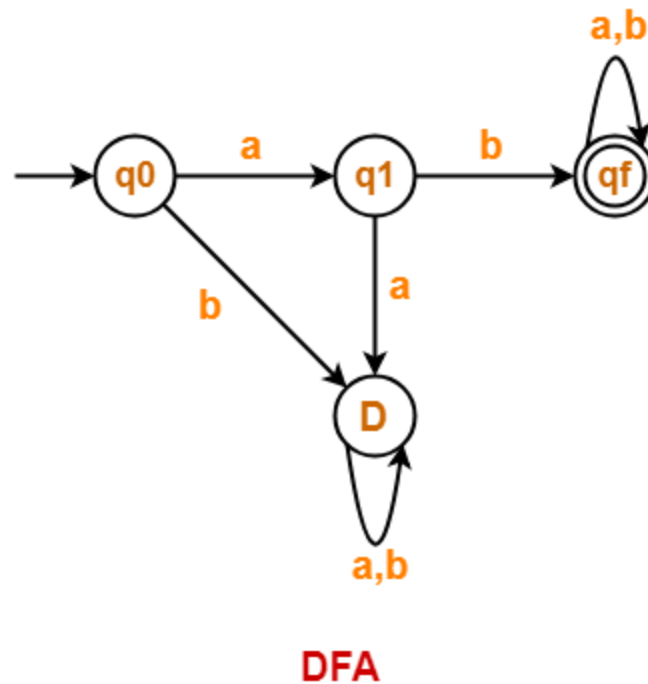
Is this a DFA?

No, but it is a *Non-deterministic Finite Automaton*

Practice problem 1

- Draw a DFA for the language accepting strings starting with 'ab' over input alphabets $\Sigma = \{a, b\}$
 - Solution:
 - **Step-01:**
 - All strings of the language starts with substring "ab". So, length of substring = 2.
 - Thus, Minimum number of states required in the DFA = $2 + 2 = 4$.
 - It suggests that minimized DFA will have 4 states.
 - **Step-02:**
 - We will construct DFA for the following strings-
 - ab
 - aba
 - abab

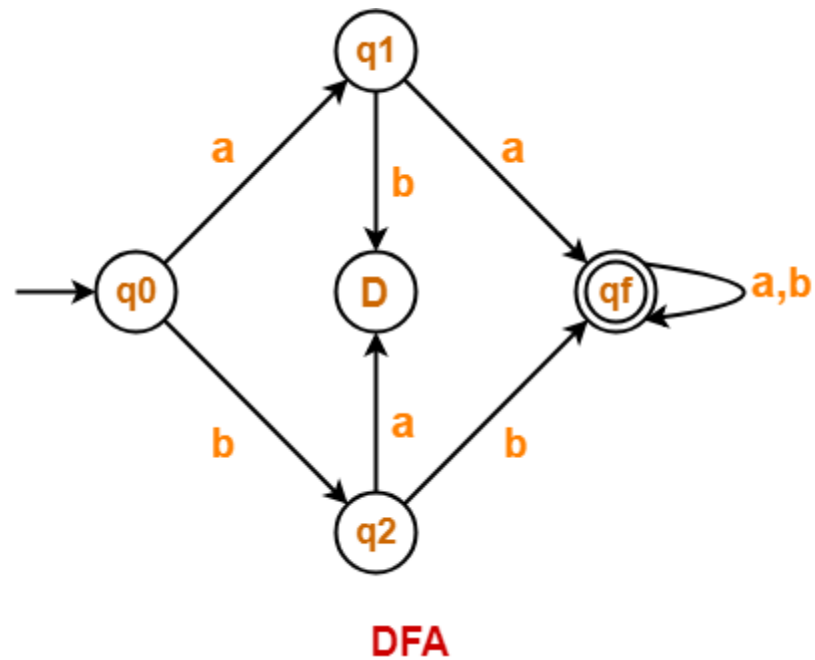
- Step 3



Practice problem 2

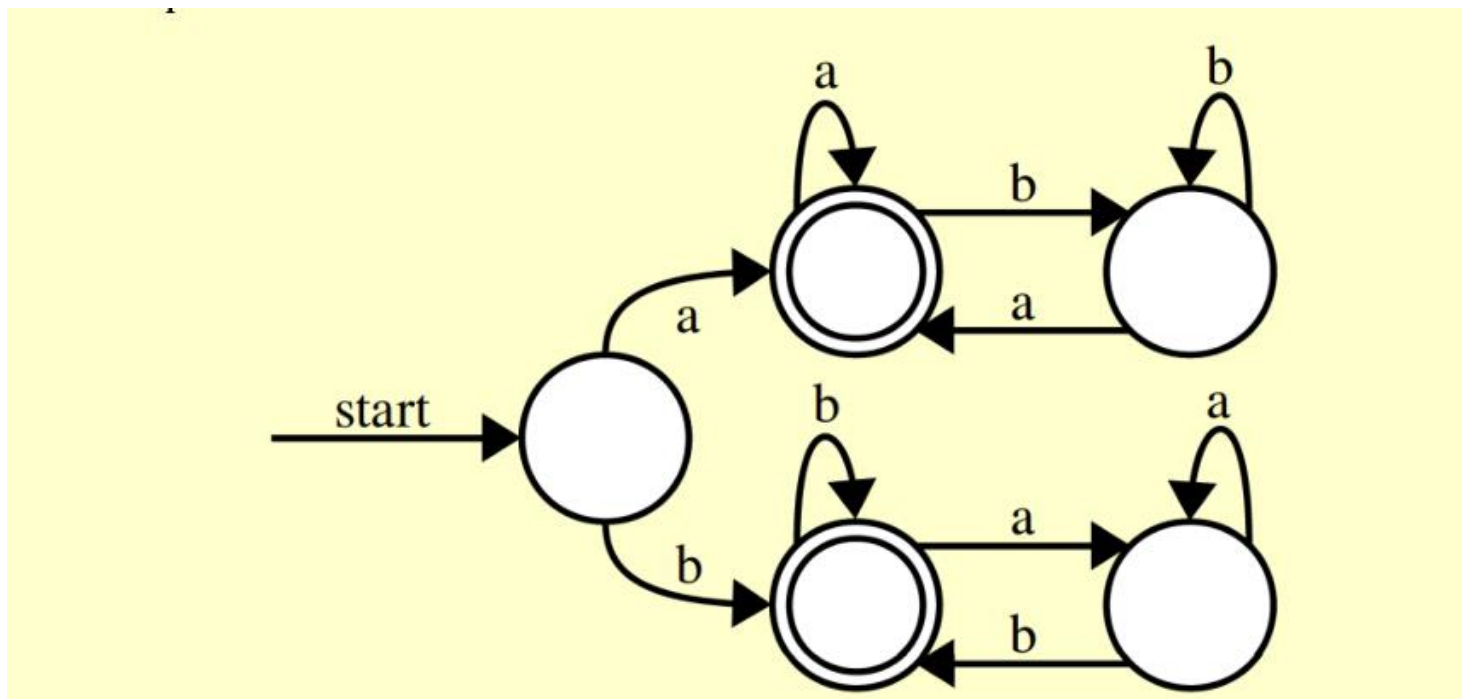
- Construct a DFA that accepts a language L over input alphabet $\Sigma = \{a, b\}$ such that L is the set of all strings starting with 'aa' or 'bb'.
- **Step-01:**
 - Minimum number of states required in the DFA = 5.
 - It suggests that minimized DFA will have 5 states.
- **Step-02:**
 - We will construct DFA for the following strings-
 - aa
 - aaa
 - aaaa
 - bb
 - bbb
 - bbbb

- Step 3



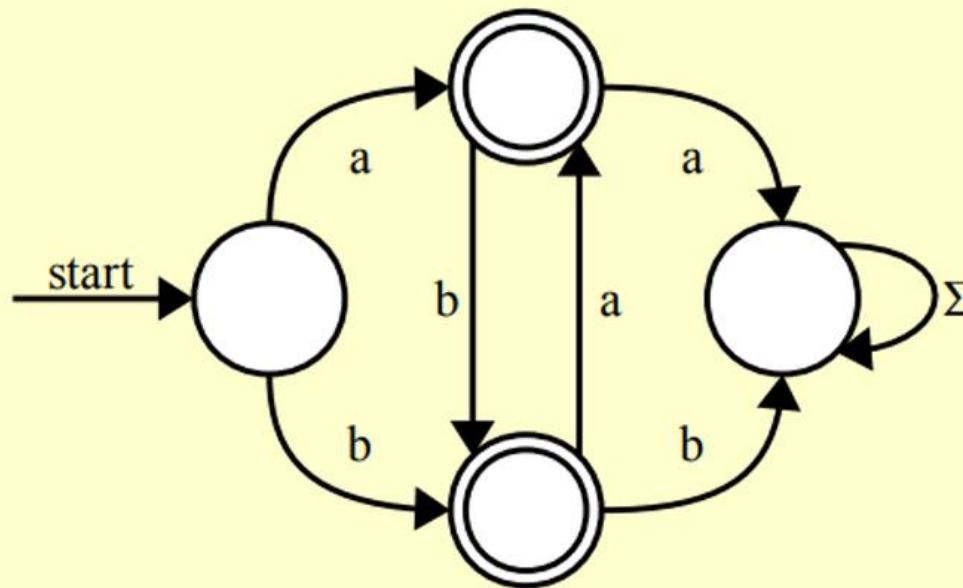
Practice problem 3

- Let $\Sigma = \{a, b\}$ and let $L = \{ w \in \Sigma^* \mid w \neq \varepsilon \text{ and the first and last character of } w \text{ are the same} \}$. Design a DFA for L .



Practice problem 4

- Let $\Sigma = \{a, b\}$ and let $L = \{ w \in \Sigma^* \mid w \text{ is a nonempty string whose characters alternate between a's and b's} \}$. Design a DFA whose language is L .



Nondeterministic Finite Automata

Nondeterminism
Subset Construction

Nondeterminism

- A *nondeterministic finite automaton* has the ability to be in several states at once.
- Transitions from a state on an input symbol can be to any set of states.

Nondeterminism – (2)

- Start in one start state.
- Accept if any sequence of choices leads to a final state.
- **Intuitively**: the NFA always “guesses right.”

Formal NFA

- A finite set of states, typically Q .
- An input alphabet, typically Σ .
- A transition function, typically δ .
- A start state in Q , typically q_0 .
- A set of final states $F \subseteq Q$.

Transition Function of an NFA

- $\delta(q, a)$ is a set of states.
- Extend to strings as follows:
- **Basis:** $\delta(q, \epsilon) = \{q\}$
- **Induction:** $\delta(q, wa) =$ the union over all states p in $\delta(q, w)$ of $\delta(p, a)$

Language of an NFA

- A string w is accepted by an NFA if $\delta(q_0, w)$ contains at least one final state.
- The language of the NFA is the set of strings it accepts.

Equivalence of DFA's, NFA's

- A DFA can be turned into an NFA that accepts the same language.
- If $\delta_D(q, a) = p$, let the NFA have $\delta_N(q, a) = \{p\}$.
- Then the NFA is always in a set containing exactly one state – the state the DFA is in after reading the same input.

Equivalence – (2)

- Surprisingly, for any NFA there is a DFA that accepts the same language.
- Proof is the *subset construction*.
- The number of states of the DFA can be exponential in the number of states of the NFA
- Thus, NFA's accept exactly the regular languages.

Subset Construction

- Given an NFA with states Q , inputs Σ , transition function δ_N , start state q_0 , and final states F , construct equivalent DFA with:
 - States 2^Q (Set of subsets of Q).
 - Inputs Σ .
 - Start state $\{q_0\}$.
 - Final states = all those with a member of F .

Critical Point

- The DFA states have *names* that are sets of NFA states.
- But as a DFA state, an expression like $\{p,q\}$ must be read as a single symbol, not as a set
- **Analogy**: a class of objects whose values are sets of objects of another class.

Subset Construction – (2)

- The transition function δ_D is defined by:
 $\delta_D(\{q_1, \dots, q_k\}, a)$ is the union over all $i = 1, \dots, k$ of $\delta_N(q_i, a)$.
- **Example:** We'll construct the DFA equivalent of our “chessboard” NFA.

Example: Subset Construction

	r	b
→ 1	2,4	5
2	4,6	1,3,5
3	2,6	5
4	2,8	1,5,7
5	2,4,6,8	1,3,7,9
6	2,8	3,5,9
7	4,8	5
8	4,6	5,7,9
* 9	6,8	5

	r	b
→ {1}	{2,4}	{5}
{2,4}		
{5}		

Alert: What we're doing here is the *lazy* form of DFA construction, where we only construct a state if we are forced to.

Example: Subset Construction

	r	b
→ 1	2,4	5
2	4,6	1,3,5
3	2,6	5
4	2,8	1,5,7
5	2,4,6,8	1,3,7,9
6	2,8	3,5,9
7	4,8	5
8	4,6	5,7,9
* 9	6,8	5

	r	b
→ {1}	{2,4}	{5}
{2,4}	{2,4,6,8}	{1,3,5,7}
{5}		
{2,4,6,8}		
{1,3,5,7}		

Example: Subset Construction

	r	b
→ 1	2,4	5
2	4,6	1,3,5
3	2,6	5
4	2,8	1,5,7
5	2,4,6,8	1,3,7,9
6	2,8	3,5,9
7	4,8	5
8	4,6	5,7,9
* 9	6,8	5

	r	b
→ {1}	{2,4}	{5}
{2,4}	{2,4,6,8}	{1,3,5,7}
{5}	{2,4,6,8}	{1,3,7,9}
{2,4,6,8}		
{1,3,5,7}		
* {1,3,7,9}		

Example: Subset Construction

	r	b
→ 1	2,4	5
2	4,6	1,3,5
3	2,6	5
4	2,8	1,5,7
5	2,4,6,8	1,3,7,9
6	2,8	3,5,9
7	4,8	5
8	4,6	5,7,9
* 9	6,8	5

	r	b
→ {1}	{2,4}	{5}
{2,4}	{2,4,6,8}	{1,3,5,7}
{5}	{2,4,6,8}	{1,3,7,9}
{2,4,6,8}	{2,4,6,8}	{1,3,5,7,9}
{1,3,5,7}		
* {1,3,7,9}		
* {1,3,5,7,9}		

Example: Subset Construction

	r	b
→ 1	2,4	5
2	4,6	1,3,5
3	2,6	5
4	2,8	1,5,7
5	2,4,6,8	1,3,7,9
6	2,8	3,5,9
7	4,8	5
8	4,6	5,7,9
* 9	6,8	5

	r	b
→ {1}	{2,4}	{5}
{2,4}	{2,4,6,8}	{1,3,5,7}
{5}	{2,4,6,8}	{1,3,7,9}
{2,4,6,8}	{2,4,6,8}	{1,3,5,7,9}
{1,3,5,7}	{2,4,6,8}	{1,3,5,7,9}
* {1,3,7,9}		
* {1,3,5,7,9}		

Example: Subset Construction

	r	b
→ 1	2,4	5
2	4,6	1,3,5
3	2,6	5
4	2,8	1,5,7
5	2,4,6,8	1,3,7,9
6	2,8	3,5,9
7	4,8	5
8	4,6	5,7,9
* 9	6,8	5

	r	b
→ {1}	{2,4}	{5}
{2,4}	{2,4,6,8}	{1,3,5,7}
{5}	{2,4,6,8}	{1,3,7,9}
{2,4,6,8}	{2,4,6,8}	{1,3,5,7,9}
{1,3,5,7}	{2,4,6,8}	{1,3,5,7,9}
* {1,3,7,9}	{2,4,6,8}	{5}
* {1,3,5,7,9}		

Example: Subset Construction

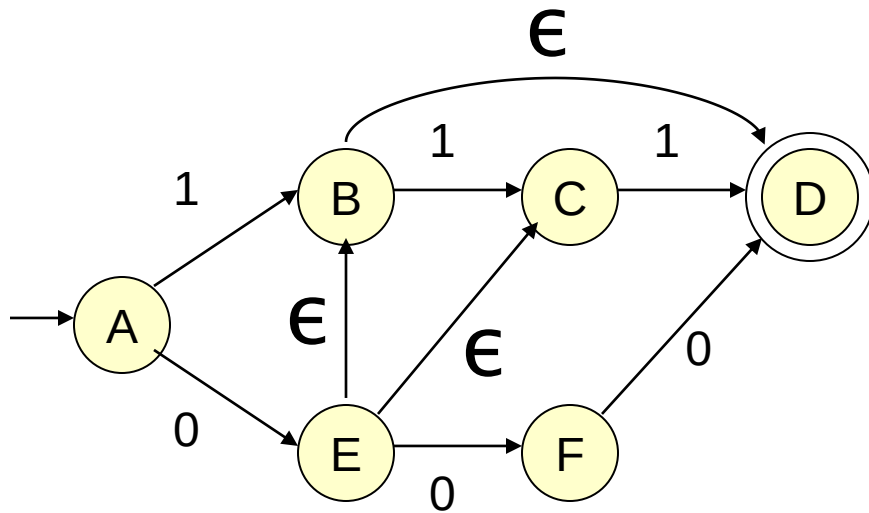
	r	b
→ 1	2,4	5
2	4,6	1,3,5
3	2,6	5
4	2,8	1,5,7
5	2,4,6,8	1,3,7,9
6	2,8	3,5,9
7	4,8	5
8	4,6	5,7,9
* 9	6,8	5

	r	b
→ {1}	{2,4}	{5}
{2,4}	{2,4,6,8}	{1,3,5,7}
{5}	{2,4,6,8}	{1,3,7,9}
{2,4,6,8}	{2,4,6,8}	{1,3,5,7,9}
{1,3,5,7}	{2,4,6,8}	{1,3,5,7,9}
* {1,3,7,9}	{2,4,6,8}	{5}
* {1,3,5,7,9}	{2,4,6,8}	{1,3,5,7,9}

NFA's With ϵ -Transitions

- We can allow state-to-state transitions on ϵ in put.
- These transitions are done spontaneously, without looking at the input string.
- A convenience at times, but still only regular languages are accepted.

Example: ϵ -NFA



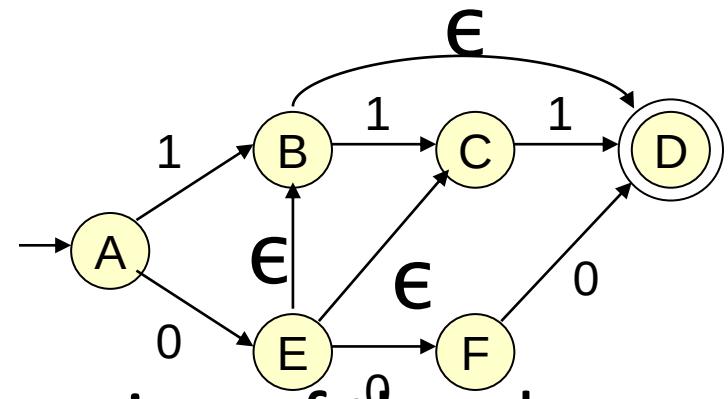
	0	1	ϵ	
→ A	{E}	{B}	$\emptyset$	
B	$\emptyset$	{C}	{D}	
C	$\emptyset$	{D}	$\emptyset$	
D	$\emptyset$	$\emptyset$	$\emptyset$	
* E	{F}	$\emptyset$	{B, C}	
F	{D}	$\emptyset$	$\emptyset$	

Closure of States

- $CL(q)$ = set of states you can reach from state q following only arcs labeled ϵ .

- **Example:** $CL(A) = \{A\}$;

$CL(E) = \{B, C, D, E\}$.



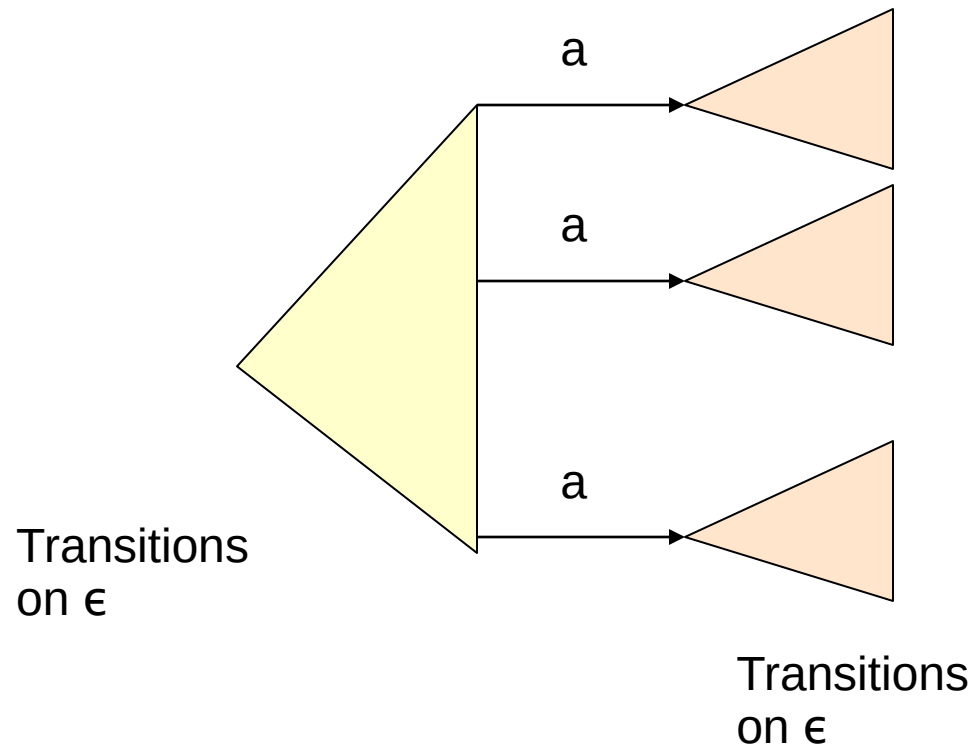
- Closure of a set of states = union of the closure of each state.

Equivalence of NFA, ϵ -NFA

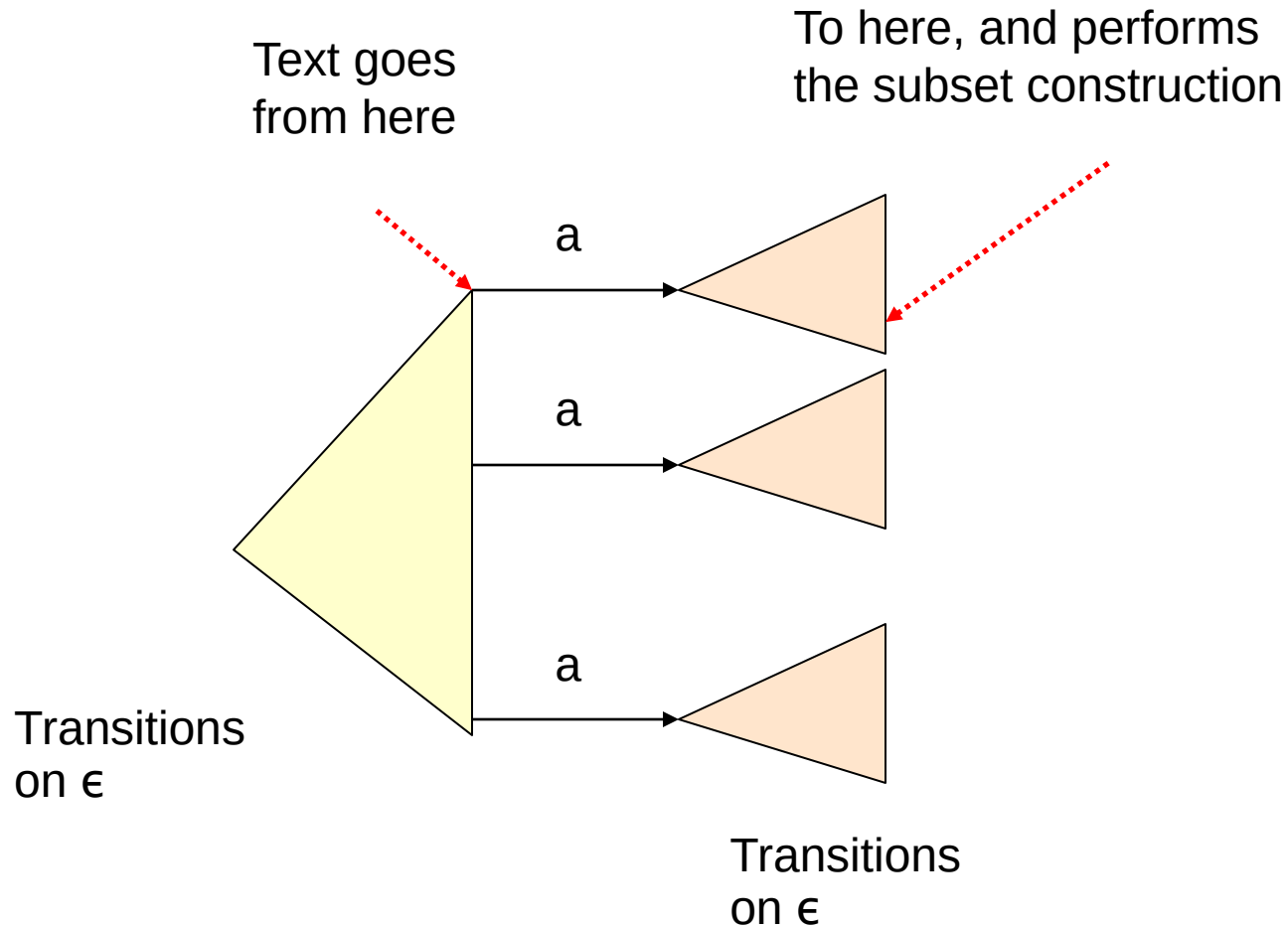
- Every NFA **is** an ϵ -NFA.
 - It just has no transitions on ϵ .
- Converse requires us to take an ϵ -NFA and construct an NFA that accepts the same language.
- We do so by combining ϵ -transitions with the next transition on a real input.

Warning: This treatment is a bit different from that in the text.

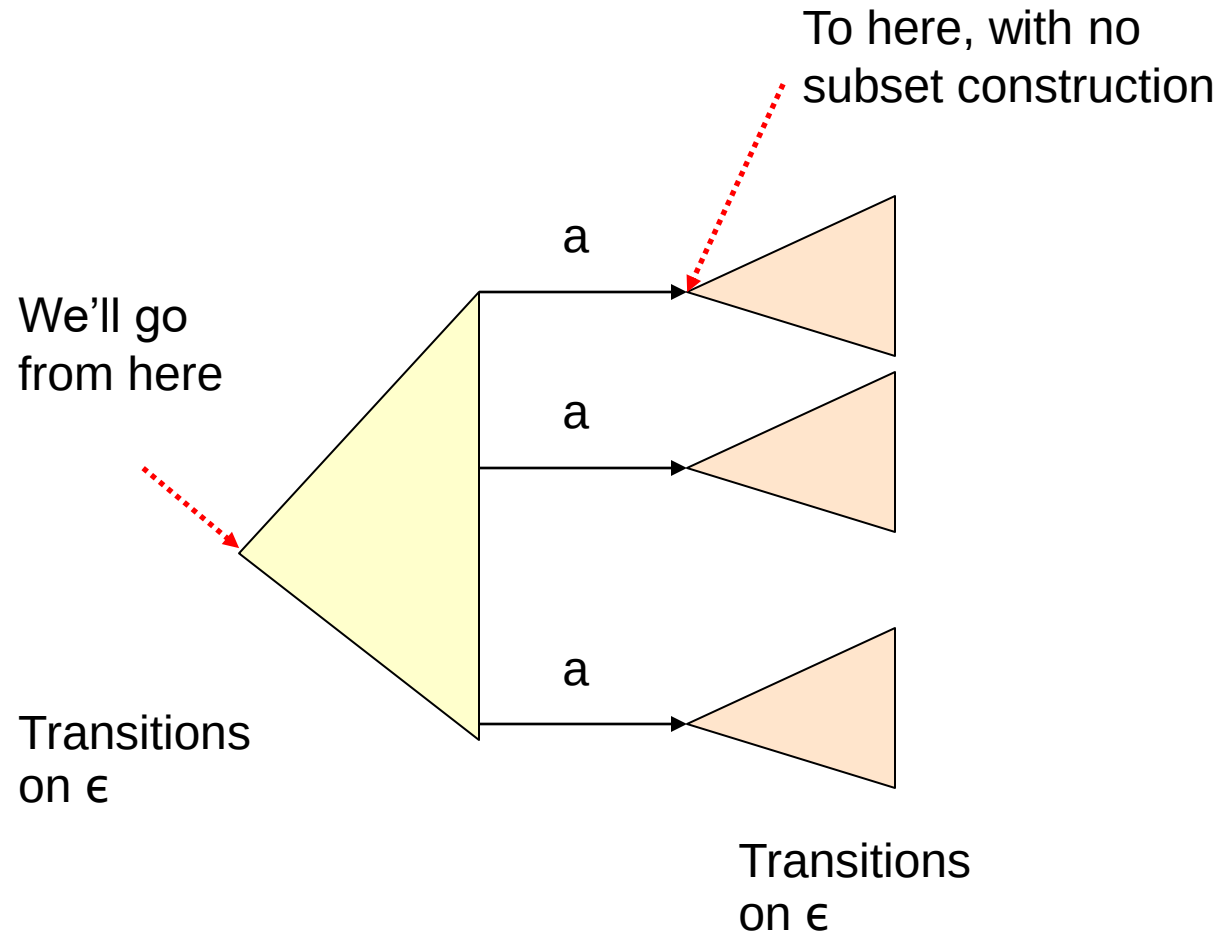
Picture of ϵ -Transition Removal



Picture of ϵ -Transition Removal



Picture of ϵ -Transition Removal



Equivalence – (2)

- Start with an ϵ -NFA with states Q , inputs Σ , start state q_0 , final states F , and transition function δ_E .
- Construct an “ordinary” NFA with states Q , inputs Σ , start state q_0 , final states F' , and transition function δ_N .

Equivalence – (3)

- Compute $\delta_N(q, a)$ as follows:
 1. Let $S = CL(q)$.
 2. $\delta_N(q, a)$ is the union over all p in S of $\delta_E(p, a)$.
- F' = the set of states q such that $CL(q)$ contains a state of F .
- **Intuition:** δ_N incorporates ϵ -transitions before using a but not after.

Interesting closures:

$CL(B) = \{B, D\};$

$CL(E) = \{B, C, D, E\}$

Example: ϵ -NFA-to-NFA

	0	1	ϵ
$\rightarrow$ A	{E}	{B}	$\emptyset$
B	$\emptyset$	{C}	{D}
C	$\emptyset$	{D}	$\emptyset$
D	$\emptyset$	$\emptyset$	$\emptyset$
* E	{F}	$\emptyset$	{B, C}
F	{D}	$\emptyset$	$\emptyset$

ϵ -NFA

Since closures of B and E include final state D.

	0	1	
$\rightarrow$ A	{E}	{B}	
* B	$\emptyset$	{C}	
C	$\emptyset$	{D}	
* D	$\emptyset$	$\emptyset$	
E	{F}	{C, D}	
F	{D}	$\emptyset$	

Since closure of E includes B and C; which have transitions on 1 to C and D.

Summary

- DFA's, NFA's, and ϵ -NFA's all accept exactly the same set of languages: the regular languages.
- The NFA types are easier to design and may have exponentially fewer states than a DFA.
- But only a DFA can be implemented!

- <https://www.gatevidyalay.com/tag/dfa-examples-with-solutions-pdf/>
- <https://web.stanford.edu/class/cs103a/assn/070S%20Solutions%20for%20Practice%20with%20Automata.pdf>
- <https://www.cse.iitd.ac.in/~naveen/courses/COL352/slides/fa3.ppt>

Thank you